

Composition Pair: Tool-Agnosticism × Linear-Cost-Scaling — Scalable Vendor-Independence as the Emergent Architectural Property in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 6, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the architectural property that emerges when two of the source paper's foundational commitments — tool-agnosticism (§7.1) and linear-cost scaling (§6) — compose, so that downstream work can reason about the composition as a single architectural object distinct from either commitment alone.

Abstract

The CKS pattern lists tool-agnosticism and linear-cost scaling among its six architectural commitments. Each is well-formed in isolation: tool-agnosticism specifies the three minimal host requirements that admit a substrate (§7.1); linear-cost scaling specifies the cost contract under which substrate growth does not produce super-linear cost on any axis except storage (§6, §6.2). Each is also independently satisfiable in ways that do not yield a deployable architecture at scale — a tool-agnostic deployment can carry super-linear cost characteristics that make scaling operationally infeasible, and a linearly-scaling deployment can buy its linearity through vendor-specific scaling features that do not survive vendor migration. This note formalizes the **composition** of the two commitments as a standalone architectural object: an emergent property — *scalable vendor-independence* — under which vendor-portability and cost-scalability hold simultaneously, with neither preserved at the expense of the other. The note states the property in four operational components, distinguishes it from each commitment in isolation, identifies what the composition forces beyond either commitment alone, enumerates anti-patterns specific to the composition, and provides an operational test with three sharpening properties.

1. Why the composition needs to be formalized as standalone

The CKS pattern names tool-agnosticism and linear-cost scaling as separate commitments and defends them in separate sections of the source paper. A reader who has internalized the two commitments individually still has work to do to see what their composition produces, because the architectural property the composition yields is not the conjunction of the two individual claims — it is a stronger property that excludes architectures satisfying each commitment singly while failing the composition.

Three motivating cases make this concrete. The first is deployments that satisfy both: a substrate migratable across hosts satisfying the three minimal requirements without architectural change, whose cost-scaling shape is a property of the substrate's own architecture rather than of any host vendor's optimizations. The second is deployments that satisfy A1.05 alone — vendor-portable in principle, but with super-linear cost characteristics that make scaling operationally infeasible on any host. The third is deployments that satisfy A1.06 alone — linearly-scaling for the current vendor only, because the linearity is achieved through vendor-specific scaling features (proprietary partition layouts, vendor-managed sharding, vendor-tuned indexing) that do not exist on alternate hosts. The first satisfies the composition; the second and third do not.

The composition is consequential because *at scale* and *across hosts* are the conditions under which AI deployments in the 2024–2026 commercial landscape must operate. Architectures that achieve scalability through vendor lock-in and architectures that achieve portability through cost-infeasibility are both visible in the field; the composition specifically forces neither pattern. The source paper's §6.2 develops the cost-curve distinction load-bearing here: linear-cost scaling is architecturally significant precisely because it is preserved as a property of the substrate's design, not bought from the host's optimizations. Composition with tool-agnosticism is what makes that property hold across hosts, not only on one of them. The strategic motivation for naming the composition as a standalone derivation is the prior-art surface area it covers: patentable derivations focused on AI deployments that scale across vendors are defensibly contested when the composition's content is publicly stated.

2. The emergent property, defined precisely

The composition of A1.05 and A1.06 yields the architectural property of **scalable vendor-independence**. The property has four operational components.

(a) Substrate operations have linear-cost characteristics independent of vendor specifics. The substrate's cost-scaling profile per A1.06 — storage and full-substrate read scale linearly with substrate size; governance, cell execution, LLM cost per execution, conflict-handling, and onboarding cost do not scale with substrate size at all — is a property of the substrate's architecture, not of the host vendor's optimizations. The same substrate hosted on any environment satisfying A1.05's three minimal requirements exhibits the same cost-scaling shape.

(b) Vendor migration does not change cost-scaling characteristics. Moving a substrate from one qualifying host to another preserves the cost contract per A1.06. The deployment's cost behavior at substrate size N is the same architectural function on the receiving host as on the sending host. Absolute cost coefficients may differ; the *shape* of the cost curve is preserved.

(c) Vendor-specific cost optimizations do not determine substrate architecture. The substrate's architecture is chosen for substrate-architectural reasons — what it represents, how cells access it, how conflicts are preserved, how rules govern cell behavior, how retraceability is carried — not for reasons that are properties of one host vendor's cost model. A substrate whose schema, partitioning, or access patterns are dictated by a particular vendor's pricing or scaling features fails this component; the architecture it has is the vendor's, not the substrate's.

(d) The deployment can grow without either vendor lock-in or super-linear cost growth. As substrate size increases, neither portability nor cost-feasibility degrades. The deployment is not forced to choose between staying on one vendor (to preserve cost-scaling) and accepting super-linear cost (to preserve portability). Both properties scale together because the architectural choices that yield each are compatible with the architectural choices that yield the other.

A deployment exhibiting all four components is scalable-vendor-independent in the CKS sense; a deployment exhibiting any three has gone partway without arriving at the composition.

3. Composition versus individual commitment

Each commitment, evaluated alone, admits architectures that fail the composition.

A1.05 alone admits vendor-portable-with-super-linear-cost. Tool-agnosticism, taken in isolation, requires only that the host satisfy three minimal requirements (persistent structured state, human read/write access, LLM access to substrate content). It says nothing about the substrate's internal access patterns or cost-scaling. A substrate whose composition cost grows quadratically or worse with substrate size satisfies A1.05 — any qualifying host can host it — but the deployment is operationally infeasible at scale on any of those hosts. Portability is preserved as a property of the host interface; the substrate cannot actually run at the scale the deployment requires.

A1.06 alone admits linear-cost-through-vendor-lock-in. Linear-cost scaling, taken in isolation, requires only that the cost-scaling profile hold; it does not specify how the profile is achieved. A substrate whose linear-cost profile depends on a specific vendor's proprietary partition manager, a specific vendor's managed sharding, or a specific vendor's caching tier satisfies A1.06 — costs do scale linearly on that vendor — but the linearity is the vendor's, not the substrate's. Migration breaks the cost contract because the receiving host has no equivalent of the proprietary mechanism the cost contract relied on.

The composition forces both properties to be properties of the substrate, not of the host. Cost-scaling per A1.06 must be a consequence of the substrate's architecture — what it stores, how it is read, how cells access it — so that any host satisfying A1.05 produces the same cost shape. Vendor-portability per A1.05 must be unconditional on the cost coefficients of any particular vendor, so that the deployment is feasible across hosts at the scale it operates. Each property is preserved by being architectural rather than vendor-architectural.

The distinction matters operationally because deployments may satisfy individual commitments while failing the composition, and the failure mode differs: A1.05-only deployments fail at scale; A1.06-only deployments fail at migration. The composition fails neither.

4. What the composition forces beyond either commitment in isolation

Six architectural decisions follow from the composition that do not follow from either commitment alone.

(a) Substrate architecture is chosen for architectural reasons, not vendor cost reasons — schema, partitioning, indexing, and access patterns are determined by the substrate's coordination role, not by

what scales cheaply on a particular vendor. (b) Cost-scaling is testable across hosts — the deployment can verify that cost behavior is the same architectural function on alternate hosts satisfying A1.05; without verification, scalable vendor-independence is a claim, not a property. (c) LLM consultations are cost-bounded within cells — LLM context window attention is typically quadratic in tokens, and if the LLM's super-linear cost determined substrate cost-scaling, the composition would fail at the LLM layer regardless of the substrate's design. The AI-as-substrate-mediator commitment (A1.04) places LLM consultations within cells, where their cost is bounded by the cell's task scope rather than by substrate size; the composition forces this bounding to be operationally enforced. (d) Vendor-specific scaling features cannot be the substrate's cost contract — proprietary partition layouts, vendor-managed sharding, and vendor-specific indexing schemes may be used by the host, but they cannot be the architectural basis of the substrate's linearity. The substrate must scale linearly through architecture, not through vendor optimizations. (e) Migration feasibility is operational, not theoretical — cost-scaling preservation, schema portability, and access-pattern equivalence are verifiable facts about candidate target hosts, not assumptions about what tool-agnosticism implies. (f) Cost characteristics are described in substrate-architectural terms — when a deployment describes its cost behavior, the description is in terms of the substrate's architecture (what cells read, how rules trigger, how conflicts are preserved), not in terms of the host vendor's billing model. The same description applies on any qualifying host.

5. Anti-patterns that violate the composition

Several anti-patterns specifically violate the composition. Some are also individual-commitment violations; the composition status is what makes the violation crisp.

LLM-context-quadratic-cost-as-substrate. When LLM context windows are used as the substrate (the anti-pattern formalized in note A3.24), the LLM's quadratic attention cost determines what is, by usage, the substrate's cost-scaling profile. A1.05 is violated because context windows do not satisfy the persistent-structured-state requirement and because the deployment is locked to whichever vendor supplies the model. A1.06 is violated because the cost profile is super-linear. The composition fails at both layers simultaneously, which is why it is the canonical severe violation.

Vendor-specific-scaling-architecture. When linear cost-scaling is achieved through proprietary vendor features (a particular database's partition layout, a particular cloud's managed sharding, a particular service's caching tier), A1.06 holds for the current vendor and fails on migration. The composition fails because vendor-portability is theoretical: migration breaks the cost contract.

Cost-scales-with-vendor-changes. When deployment cost characteristics depend on vendor-specific pricing structure (per-query, per-data-volume, per-compute-time) such that migration changes the cost-scaling shape even with architectural functionality preserved, the composition fails. The substrate is portable in function but not in cost behavior.

Black-box-agent-memory-cost-determining-architecture. When a vendor-specific agent memory system (the anti-pattern class formalized in A3.25) supplies cost characteristics that the substrate's architecture is built around, the deployment is locked to that vendor's cost model. A different vendor's agent memory has different scaling characteristics, breaking the composition.

Adjacent-component-as-substrate-substitute, when the substitute has super-linear cost. When an adjacent AI component (vector index, retrieval engine, embedding store) is used as a substrate substitute (the anti-pattern class formalized in A3.23) and the substitute carries super-linear retrieval cost, the deployment fails A1.06 even when A1.05 holds — and may fail A1.05 too, if the substitute is vendor-specific.

Proprietary-database-feature-as-substrate-architecture. When a substrate's schema or access pattern is structured around a proprietary database feature, substrate architecture has been chosen for vendor reasons. The composition fails at component (c).

Cost-optimization-as-architectural-decision. When cost optimization for the current vendor drives architectural choices not preserved on migration (e.g., denormalizing for one vendor's pricing, partitioning for another's), the architecture has shaped to the vendor and cannot be carried unchanged to a different host.

6. What the composition is NOT

The composition is not vendor-portability without cost-feasibility. A deployment whose substrate is portable in principle but whose cost-scaling makes it infeasible at scale satisfies A1.05 but not the composition. A claim of "we can migrate" is not a claim of "we can migrate and operate at scale."

The composition is not cost-linearity through vendor-specific optimization. A deployment whose linear-cost shape depends on vendor-specific scaling features satisfies A1.06 for the current vendor but not the composition. A claim of "our cost is linear" tied to a specific host is not a claim of "our cost is linear across hosts."

The composition is not cost-scaling that is "good enough" for the current scale and vendor but degrades at migration or under growth. The composition is preserved as a property of the architecture, not as a contingent operational fact about the current deployment.

The composition is not theoretical portability without operational migration testing. A deployment that has never been migrated can claim portability in principle; the composition's claim is that migration preserves the cost contract, and the only way to confirm this is to verify it on candidate target hosts.

7. Why the composition is load-bearing

The composition supports the part of the CKS pattern that distinguishes it from architectures that satisfy each commitment alone but fail their joint consequence. Tool-agnosticism alone makes a substrate hostable in commodity environments; linear-cost scaling alone makes a substrate growable to coordination-scale volumes. Their composition makes a substrate growable to coordination-scale volumes *while* hostable in commodity environments — the property required for deployments where both axes are alive at once.

The composition also constrains how AI-as-substrate-mediator (A1.04) is realized. The LLM's super-linear attention cost is bounded inside the cell, so that the LLM does not propagate its cost characteristics to substrate operations. Without this bounding, substrate growth would be governed by the LLM's quadratic cost rather than by the substrate's linear cost, and the substrate's tool-agnosticism

would coincide with the LLM's vendor lock-in.

The composition also supports A1.13's composition requirements. Deployments choose composition partners — adjacent components that supply input, derived views, or separate concerns — based on architectural fit. Scalable vendor-independence is what makes the choice reversible: a partner that carries vendor lock-in or super-linear cost can be swapped without re-architecting the substrate.

8. Operational test

A deployment instantiates the $A1.05 \times A1.06$ composition if and only if all four operational components in §2 hold and the following three sharpening properties are observable.

(e.1) Vendor-migration-cost-preservation. The deployment can be migrated from one host satisfying the three minimal host requirements to another, and the cost-scaling shape is the same after migration as before. Absolute coefficients may differ; the architectural function relating cost to substrate size, rule variety, and intervention frequency is unchanged. The property is verifiable in practice: a candidate migration is performed (or simulated against a representative target host), and cost behavior is measured against the source host's behavior at the same substrate size. If the shape changes, the composition is violated.

(e.2) Substrate-cost-architecture-independence. The substrate's cost-scaling characteristics are derivable from the substrate's architecture — its schema, access patterns, cell-scope contracts, conflict-preservation structure — without reference to any specific vendor's optimization features. If the cost contract requires citing proprietary partitioning, managed sharding, vendor-specific indexing, or any other vendor-architectural mechanism to be defended, the architecture has not been chosen independently of the vendor, and the composition is violated.

(e.3) LLM-cost-bounded-within-cell. Each LLM consultation occurs within a cell whose task scope bounds the consultation's input and output, such that the LLM's super-linear attention cost does not propagate to substrate operations. Substrate cost-scaling can be computed without reference to LLM cost; LLM cost can be computed without reference to substrate size. If LLM consultations span the substrate (e.g., context windows holding substrate content directly), the bounding is lost and the composition is violated.

A deployment passing all three sharpening properties — alongside the four operational components in §2 — is scalable-vendor-independent in the CKS sense.

One-sentence test. *Can the deployment be migrated to a different host satisfying the three minimal host requirements, with cost-scaling shape preserved, without re-architecting the substrate?* If yes, the composition is instantiated; if no, it is not.

9. Conclusion

The composition of A1.05 and A1.06 produces an architectural property — scalable vendor-independence — that neither commitment yields alone. The composition forces vendor-portability and cost-scalability to be properties of the substrate's architecture rather than of any particular host's optimizations, so that the deployment can grow at scale and can move across hosts

without trading either property for the other. The composition specifically excludes architectures that satisfy each commitment singly while failing their joint consequence: vendor-portable-with-super-linear-cost, linear-cost-through-vendor-lock-in, LLM-context-as-substrate, vendor-specific-scaling-architecture, and the related anti-patterns enumerated above.

Naming the composition as standalone — distinct from each commitment's individual decomposition (the A2.24–A2.28 line for tool-agnosticism, the A2.29–A2.34 line for linear-cost scaling) and from the individual-commitment anti-patterns (A3.24, A3.25, A3.23) — makes the architectural object available to subsequent work as a single citable property. The next composition pairs in Phase A4 formalize additional joint architectural consequences along the same lines: tool-agnosticism with substrate-as-source-of-truth, AI-as-substrate-mediator with the determinism contract, the substrate-cell boundary with the determinism contract, and others. Each pair is an architectural object in its own right, derivable from but not reducible to the commitments it composes.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Composition Pair: Tool-Agnosticism × Linear-Cost-Scaling — Scalable Vendor-Independence as the Emergent Architectural Property in the Coordination Knowledge Substrate Pattern*. May 6, 2026. ORCID: 0009-0004-8065-3235.